Report Practical 1

Algorithms And Data Structures Practical One

On finding the maximum amount of roads that can be closed whilst keeping every stage of the Vierdaagsefeesten reachable by either means of transport.

Group: 21
By:

- Daniël Groenendijk, **s1169129**
- Dirk van Roosmalen, **s1176271**

Contents

---

1. Algorithm Explanation

1.1 Initialization

During the initialization phase, the algorithm takes the given input file and loops through every line, making sure to separate the very first line as it's a bit special. The only thing of use that this line contains is the total amount of stages, while the total number of roads between them is instead retrieved in the next part.

In this section of the initialization the remaining lines of the file are counted and transformed into an array of integers, where entry index 0 and index 1 represent the origin and destination of the road while entry index 2 represents the type of road.
During the remaining execution of the algorithm, the acquired count of roads is used and preferred over the one given on the first line due to the fact that this number is certain to be accurate, while this can't always be said for the given number.

1.2 Algorithm Logic

During the execution state of the algorithm, two disjoint sets are used as data structures to represent the networks of roads for both traffic types. These sets each contain the total

amount of stages present in the input file and are each used to figure out which stages haven't already been reached, where the lack of weight or a requirement for the shortest path ensures that once a path is found, it is correct.

The roads are assessed in two loops, where one is used on all the roads that allow both modes of transportation and the other one on those that don't. In each loop it is checked if adding any of the roads looped through actually adds a new previously unreachable stage to the network. If this is the case, the road will be used and the count of used roads is increased by one. If on the other hand, a road doesn't connect to a new stage, we have no reason to connect it to our network and is therefore discarded.

Once both loops have finished, a final check is done on the disjunction sets to see if both of them still have the full number of stages reachable.

1.3 Returning the Answer

For formulating the final answer, the combination of the total amount of roads and the used amount of roads is used where the latter is subtracted from the former to get the total amount of roads that could be removed from the road network. This is based on the fact that the difference between the total roads and used roads must be the number of roads that are not used, thus being the number of roads that could be removed.

One exception to the above explanation occurs when the final check mentioned in chapter 1.2 fails, as this implies that not all stages are reachable anymore. In this case, the algorithm instead returns a -1 to signal this failure.

1.4 Optimizations

While the cornerstone of this algorithm relies on keeping a number of disjoint sets equal to the types of road users, a process that is fairly optimized by design due to the relative simplicity of the data structure, some optimizations were still made in an attempt to improve performance.

The first is the simplest, one that is also needed in part for always attaining correctness. By looking through the types of roads that allow multiple users first, the desired amount of stages can be reached with often times less roads, meaning that subsequent loops over the remaining roads don't need to find as many stages as they'd otherwise need to. This is also needed for finding the optimal number of roads to remove, as a combination of a bus and pedestrian road might otherwise be used while one that allows for both would've sufficed as well. This is a minor reduction to average runtime.

The second optimization is born from the realization that all roads are looped through during initialization, at which point their types are already known. Combine this with how the disjoint set also loops over each road whilst only caring about the respective types of roads that they're looking for; so-called twin roads or single roads. From this it is then possible to separate roads into different sets during initialization. By doing this, the algorithm only has to

loop through those subsets of roads once, instead of going through the full set multiple times.

This second optimization has a far greater impact, where its effect can best be described as reducing the runtime of that piece of the algorithm from $O(\text{Road Types} \cdot \text{Roads})$ to just $O(\text{Roads})$, which is very impactful when additional types of roads would be added or the number of roads increases.

Third and last optimization is the usage of a ranked disjoint set,
where the ranked part refers to how the disjoint set will keep its internal tree balanced.
This balancing act ensures that, the operational cost of the `union` operation on the disjoint set goes from $O(n)$ to $O(log(n))$.
When further enhanced with path compression during the `find` operation this can be further reduced to $O(\alpha(n))$,
which can almost be seen as $O(1)$ in practice, a major improvement over the $O(n)$ from the beginning.

---

2. Correctness Analysis

The correctness of the algorithm relies on the below proof holding for any reasonable input, i.e. the sample sets provided on DomJudge.

2.1 Correctly identifying new stages

The algorithm represents the network of roads in the graph as a spanning tree, one which by definition is acyclic and contains a single root node. The root node of a tree is shared by any member of that tree. From this it follows that a given node that does not share this same root, is not part of that tree. By assessing the roots of the nodes on both sides of the a new edge, we can determine whether this road connects to a new stage or tree, or is simply a different road to an already reachable stage from the current tree. In the implementation, any disconnected node or group of nodes is represented as a disjoint set.

2.2 Optimal solution for single modes of transport

The goal of removing the maximum number of roads is keeping the minimum amount of roads to keep all stages reachable. To connect n nodes in a given undirected graph, would take at least $n - 1$ total edges, which is the lower bound of roads kept. Since the graph is unweighted and the lengths of the roads are thus irrelevant, the algorithm iteratively checks whether or not any road connects to a previously unreachable stage or disjoint set of stages for a single mode of transport. If it does, the stage or set of stages is connected to the tree, as connecting that new stage using a single road must be the most optimal solution.

Assuming every stage is reachable, the greedy approach of starting from a given node and connecting any new stage or disjoint set of stages will always yield the optimal solution by

connecting n stages using $n-1$ roads.

2.3 Optimizing for multiple modes of transport

The problem with assessing single modes of transport at a time is that although producing locally optimal solutions, these don't necessarily hold as a globally optimal solution, as these locally optimal solutions could use completely differing sets of roads. This is because these solutions don't favor roads that offer multiple types of transportation. These types of roads are potentially more efficient than single transport type ones are, as these allow for usage at no additional 'cost' for local solutions if utilized by earlier local solutions. This could lead to a 'more than optimal' local solution below the aforementioned local lower bound. To ensure maximum usage of these types of roads, the algorithm classifies each road and assesses these multi-traffic type roads first, ensuring that they are optimally used.

2.4 Calculating the number of roads removed

For each connection (or union) between (a group of) stages, the number of used roads which is initialized at 0, is increased by 1. The total amount of roads that could be removed must be $n_{removed} = n_{total} - n_{used}$, where $n_{total} = n_{twinroads} + n_{singleroads}$.

2.5 Detecting unreachable stages

In some cases is possible for there to be no solution, as one or more stages could be unreachable via one or either modes of transport. Upon initialization, the algorithm takes note of the total amount of stages it now expects to find. When it is done assessing every road, it checks if the total amount of disjoint sets left for either method of transport is 1. If it isn't, it must mean that one or more (groups of) stages are not connected with, and are thus unreachable from, the first stage.

---

3. Complexity Analysis

The time complexity of the algorithm is the sum of the time complexity of the initialization and the analyzation phases of the algorithm. Note that this assumes a single file as input where $\|V\|$ represents the number of stages and $\|E\|$ the number of roads present.

3.1 Initialization

Functionality	Complexity
Reading and parsing the file.	$O(\|E\|)$
Creating a disjoined set for each stage.	$O(\|V\|)$
Categorizing the roads.	$O(\|E\|)$

3.2 Analyzation

Functionality	Complexity
Single `DisjointSet.find` operation.	$O(\|\|V\|\|)$
Single `DisjointSet.union` operation.	$O(2 \cdot \|\|V\|\|)$
Attempt to union all disjoint sets of stages reachable by roads available for **both** modes of travel.	$O(\|\|E_{both}\|\| \cdot \|\|V\|\|)$
Attempt to union all disjoint sets of stages reachable by roads available for **a single** mode of travel.	$O(\|\|E_{single}\|\| \cdot \|\|V\|\|)$
Checking if all stages are connected.	$O(1)$

3.3 Total complexity

The total time complexity of the initialization phase comes out to $O(\|\|V\|\| + \|\|E\|\|)$.
The total time complexity of the analyzation phase comes out to $O(\|\|V\|\| \cdot \|\|E\|\|)$.
For the entire algorithm, the time complexity also comes out to be $O(\|\|V\|\| \cdot \|\|E\|\|)$.

---

4. Reflection

During the design, implementation and testing of this algorithm our group ran into a couple of challenges. First of which was the the implementation the of efficient disjoint set in our final solution, a data structure that received relatively little attention during the lectures. Since the primary focus of these lectures regarded versions of BFS, DFS and Dijkstra, our first attempt also tried using these algorithms. This attempt revolved around a modified BFS that made a pair of min span trees, based on the theory that these would have the minimal amount of edges to connect al the vertices, thus making their total edge count equal to the minimal amount of roads we would need. Since both min span trees could contain the same road, there was a problem with this attempt however as it would count these twice and thus arrive at an incorrect count.

During our attempt to fix this, we eventually stumbled upon the disjoint set structure, which after some contemplation proved to be perfect for checking if using a given road would be beneficial for the graph or not. This, combined with other optimizations which used the disjoint set, became our final solution.

Another challenge we faced came in the form of getting the inputs right, as both our local testing solution and the DomJudge environment proved to be difficult to work with when testing our attempt. The first part of this challenge came from how the `.in` files downloaded from DomJudge for local testing were inconsistent in their usage of `\n` characters, thus requiring a small refactor of the way the algorithm separates the input from a line based solution to one that only looks at the numbers.

The bigger issue came from testing the algorithm on DomJudge itself, as the minimal documentation and feedback given by the environment made it fairy difficult to understand what our program was prompted with and in which way the input was presented, thereby requiring a small dozen of attempts before the input from DomJudge was correctly interpreted by our algorithm. Providing more explanation or documentation as to how DomJudge calls a provided program would thus be our main feedback in regards to this assignment, as a large amount of resources could've been saved because of this.

Apart from these issues, the remainder of the project was a good mixture between challenging and intriguing, as the brainstorming phases about how we could break down, translate and solve the problem from the assignment were enjoyable and did not only move us towards the solution but also provided new insights about unrelated topics.